ASIAN INSTITUTE OF TECHNOLOGY
School of Engineering and Technology
Department of Data Science and Artificial Intelligence

CHIANGMAI AIR QUALITY ALERT SYSTEM: END-TO-END MLOPS PIPELINE FOR PM2.5 FORECASTING

Final Report

by

Supanut Kompayak (st126055)
Shuvam Shrestha (st125975)

A Final Report Submitted in Partial Fulfillment of the Requirements
for the Course AT82.9002 – Selected Topic: Data Engineering and MLOps

Instructor:	Dr. Chantri Polprasert
Course:	AT82.9002 Selected Topic: Data Engineering and MLOps
Semester:	January 2026

Asian Institute of Technology, Thailand
April 2026

Author's Declaration

We, Supanut Kompayak and Shuvam Shrestha, hereby declare that this report entitled **"CAAS: ChiangMai Air Quality Alert System — An End-to-End MLOps Pipeline for PM2.5 Forecasting"** is the result of our own work carried out as the final capstone project for the course **AT82.9002 Data Engineering and MLOps** at the Asian Institute of Technology (AIT) during the January 2026 semester.

We further declare that:

- All external data sources (Pollution Control Department Thailand, Open-Meteo, NASA FIRMS) are publicly available and have been cited in the References section.
- All third-party software libraries (LightGBM, XGBoost, TensorFlow, scikit-learn, MLflow, Evidently, FastAPI, Streamlit, Terraform, Docker, and GitHub Actions) are used under their respective open-source licenses and have been acknowledged.
- All figures, tables, experimental results, and source code presented in this report were produced by the authors unless explicitly attributed otherwise.
- This work has not been submitted, in whole or in part, for any other degree or academic qualification.

Supanut Kompayak (st126055) Shuvam Shrestha (st125975)

Date: April 22, 2026

Asian Institute of Technology
School of Engineering and Technology
Department of Data Science and Artificial Intelligence

Acknowledgments

We wish to express our sincere gratitude to the individuals and organisations whose support made this capstone project possible.

First and foremost, we thank the course instructors and teaching assistants of **AT82.9002 Data Engineering and MLOps** at the Asian Institute of Technology for their guidance throughout the semester. The continuous feedback on our proposal and progress report shaped the scope, rigour, and engineering discipline of the final CAAS system. Specific feedback on drift-monitoring policy design, cost realism, and contribution framing was instrumental in strengthening the final report.

We gratefully acknowledge the data providers whose open access made this work feasible: the **Pollution Control Department (PCD) of Thailand** for publishing historical PM2.5 records through the air4thai portal; **Open-Meteo** for providing free ERA5-quality meteorological reanalysis through a well-documented API; and **NASA FIRMS** for making near-real-time satellite fire hotspot data available under a permissive research licence. Without these three sources, a self-contained, reproducible end-to-end pipeline would not have been possible at the student scale.

We thank the open-source maintainers of the libraries and services on which CAAS is built — LightGBM, XGBoost, TensorFlow, scikit-learn, Optuna, MLflow, Evidently, FastAPI, Streamlit, Terraform, Docker, and GitHub Actions — whose collective work compresses the distance between a research prototype and a production-grade system into a matter of weeks of student effort.

Finally, we thank our families, classmates, and peers at AIT for their patience and encouragement during long debugging sessions, cloud provisioning failures, and the inevitable late-night rebuilds that every real engineering project entails.

Supanut Kompayak (st126055)
Shuvam Shrestha (st125975)
Asian Institute of Technology, April 2026

Abstract

Chiang Mai, Thailand experiences severe annual PM2.5 pollution events driven by agricultural burning, with concentrations frequently exceeding the WHO 24-hour guideline of 15 µg/m^3 by an order of magnitude. This report presents CAAS (ChiangMai Air Quality Alert System), a production-grade end-to-end MLOps pipeline for multi-horizon PM2.5 forecasting and binary hazard alerting (PM2.5 > 50 µg/m^3). CAAS integrates three data sources — PCD Thailand PM2.5 measurements, Open-Meteo ERA5 weather reanalysis, and NASA FIRMS satellite fire hotspot data — into a 45-feature daily time-series matrix spanning 2011–2025. A LightGBM champion model outperforms an XGBoost strong-secondary and an LSTM comparison model at all forecast horizons, achieving MAE of 5.12, 6.77, and 8.25 µg/m^3 at t+1, t+3, and t+7 days respectively (R^2 = 0.841 / 0.729 / 0.600, AUROC = 0.992 / 0.973 / 0.946 on the 2024–2025 test set). SHAP analysis reveals a physically interpretable shift in dominant features: recent PM2.5 persistence (t+1), 14-day fire accumulation (t+3), and seasonal calendar position (t+7). An ablation study quantifies the measurable contribution of the NASA FIRMS features (+4.3% to +6.8% MAE when removed), and alert-threshold tuning confirms the default 50 µg/m^3 WHO/Thai threshold is near-optimal for t+3 and t+7 horizons. The MLOps stack — MLflow experiment tracking, FastAPI inference, a seasonal-aware Evidently drift monitor, and Terraform-provisioned AWS infrastructure orchestrated by GitHub Actions — constitutes a deployable, self-retraining system operating at approximately $0.55/day.

Keywords: PM2.5 forecasting, air quality, MLOps, LightGBM, XGBoost, LSTM, SHAP, drift monitoring, Evidently, MLflow, FastAPI, Terraform, AWS, NASA FIRMS, Chiang Mai.

Contents

Author's Declaration i

Acknowledgments ii

Abstract iii

List of Tables vii

List of Figures viii

List of Abbreviations ix

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Questions . . . 2
- 1.4 Objectives . . . 2
- 1.5 Scope and Contributions . . . 3
- 1.6 Organisation of the Report . . . 3

2 Literature Review 5
- 2.1 Air-Quality Forecasting for PM2.5 . . . 5
- 2.2 Gradient Boosting Versus Deep Learning for Tabular Time Series . . . 6
- 2.3 Satellite Fire Observations in Atmospheric Modelling . . . 6
- 2.4 MLOps Patterns for Environmental Machine-Learning Systems . . . 7

3 Data Engineering 8
- 3.1 Data Sources . . . 8
- 3.2 Data Ingestion Pipeline . . . 8
- 3.3 Feature Engineering . . . 9
- 3.4 Data Storage and Versioning . . . 10
- 3.5 Pipeline Orchestration . . . 10

4 Machine Learning Model Development 11
- 4.1 Dataset and Splits . . . 11
- 4.2 Baseline . . . 11

4.3 Model Selection . . . 12
4.4 Results . . . 12
4.4.1 LightGBM (Champion) . . . 12
4.4.2 XGBoost (Strong Secondary) . . . 12
4.4.3 LSTM (Comparison) . . . 13
4.4.4 Actual vs. Predicted . . . 14
4.5 Ablation Study: FIRMS Fire Data . . . 14
4.6 Alert Classifier Analysis (Scenario C) . . . 15
4.7 Model Interpretation (SHAP) . . . 16

5 MLOps Implementation **18**
5.1 System Architecture . . . 18
5.2 Experiment Tracking and Model Registry . . . 18
5.3 Model Serving . . . 19
5.4 CI/CD Pipeline . . . 20
5.5 Monitoring and Drift Detection . . . 22
5.6 Infrastructure as Code . . . 23

6 Performance Analysis **24**
6.1 Model Performance Summary . . . 24
6.2 Inference Latency . . . 24
6.3 Scalability Analysis . . . 25
6.4 Cost Analysis . . . 25

7 Conclusion and Recommendations **27**
7.1 Conclusions . . . 27
7.1.1 Gradient-Boosted Trees Outperform LSTM at All Horizons . . . 27
7.1.2 NASA FIRMS Fire Data Adds Measurable Value . . . 27
7.1.3 The Predictive Signal Shifts With Horizon . . . 28
7.1.4 The Default 50 $\mu g/m^3$ Alert Threshold Is Near-Optimal . . . 28
7.1.5 The MLOps Pipeline Is Production-Ready Within Its Design Scope . . . 28
7.2 Recommendations . . . 28
7.2.1 For Operational Adoption . . . 28
7.2.2 For Educational Adoption . . . 29
7.3 Future Research Directions . . . 29
7.3.1 Multi-Station Expansion . . . 29
7.3.2 Weather Forecast Integration . . . 29
7.3.3 Improved Long-Horizon Modelling . . . 29
7.3.4 Seasonal-Component Modelling for Sparse-Fire Periods . . . 29
7.3.5 Kubernetes Serving at Scale . . . 30
7.3.6 Deeper Hyperparameter Optimisation . . . 30
7.3.7 Limitations . . . 30

References **31**

A Reproduction Guide **34**
A.1 Prerequisites . . . 34
A.2 Step 1 — Clone and Environment Setup . . . 34
A.3 Step 2 — Configure Environment Variables . . . 34
A.4 Step 3 — Provision AWS Infrastructure with Terraform . . . 35
A.5 Step 4 — Historical Data Backfill . . . 35
A.6 Step 5 — Train Models and Register with MLflow . . . 35
A.7 Step 6 — Run Analysis Scripts (Optional) . . . 36
A.8 Step 7 — Run Inference and Upload to S3 . . . 36
A.9 Step 8 — Build and Deploy Docker Container . . . 36
A.10 Step 9 — Enable CI/CD Automation . . . 37
A.11 Step 10 — Launch the Streamlit Dashboard . . . 37
A.12 Step 11 — Run the Test Suite . . . 37

B Full Metric Tables **38**
B.1 LightGBM (Champion) — Validation and Test Metrics . . . 38
B.2 XGBoost (Strong Secondary) — Validation and Test Metrics . . . 38
B.3 LSTM — Validation and Test Metrics . . . 39
B.4 Alert Classification Metrics (LightGBM Champion, 50 µg/m^3 threshold) . . . 39
B.5 SHAP Mean Absolute Values — Top 5 Features per Horizon (XGBoost basis) . . . 40

VITA **41**

List of Tables

3.1 CAAS Feature Groups 9

4.1 Train / Validation / Test Split 11
4.2 Persistence Baseline — Test MAE 11
4.3 LightGBM Test Set Performance 12
4.4 XGBoost Test Set Performance (Optuna-tuned) 12
4.5 LSTM Test Set Performance 13
4.6 Scenario D — MAE Impact of Removing FIRMS Features (XGBoost) 14
4.7 Scenario C — Alert Threshold Optimisation (XGBoost scores) 15

5.1 CAAS API Endpoints 20

6.1 Full Model Comparison — Test Set 24
6.2 Estimated Monthly Operating Cost (AWS ap-southeast-1) 25

B.1 LightGBM Champion Full Metrics (Validation and Test Sets) 38
B.2 XGBoost Secondary Full Metrics (Validation and Test Sets) 38
B.3 LSTM Full Metrics (Validation and Test Sets) 39
B.4 Binary Alert Classification Performance on Test Set (LightGBM Champion) . . . 39
B.5 Top-5 SHAP Features by Horizon (Mean |SHAP| on Test Set) 40

List of Figures

3.1 CAAS daily data pipeline — from ingestion to drift monitoring. 9

4.1 Champion t+1 actual vs. predicted PM2.5 on the 2024–2025 test set. The model captures seasonal haze peaks accurately. 14
4.2 MAE comparison: full model vs. model without FIRMS fire features. 15
4.3 Precision-recall curve and F1 vs. threshold for t+1 alert classifier. 15
4.4 SHAP summary plot at t+1. `pm25_lag1` dominates, confirming that next-day PM2.5 is primarily driven by today's concentration. 16
4.5 SHAP summary plot at t+3. `hotspot_14d_roll` becomes the dominant signal, reflecting fire accumulation over two weeks. 17

5.1 CAAS cloud architecture on AWS. Three external APIs (air4thai, Open-Meteo, NASA FIRMS) feed a GitHub Actions-orchestrated ingestion layer (3-hourly cron) into four S3 tiers (Raw, Processed, Feature Store, Model Artifacts). A single EC2 t3.small instance hosts FastAPI, MLflow, and Streamlit. Evidently AI drift monitoring runs as a daily scheduled job. 18
5.2 MLflow experiment view showing CAAS-LGBM-Optuna champion runs alongside the XGBoost and LSTM comparison families. 19
5.3 CAAS Streamlit dashboard showing PM2.5 forecast cards and history. 20
5.4 Champion/challenger model promotion flow with validation gate. 22
5.5 Evidently drift report output (2026-04-03). Core drift detected in `pm25_lag1` and `pm25_roll7_mean` during December 2025 — expected seasonal behaviour, not model failure. 23

List of Abbreviations

Abbreviation	Meaning
AIT	Asian Institute of Technology
API	Application Programming Interface
AUROC	Area Under the Receiver Operating Characteristic curve
AWS	Amazon Web Services
CAAS	ChiangMai Air Quality Alert System
CI/CD	Continuous Integration / Continuous Deployment
CSV	Comma-Separated Values
DE	Data Engineering
EC2	Amazon Elastic Compute Cloud
ECMWF	European Centre for Medium-Range Weather Forecasts
ERA5	Fifth-generation European Reanalysis (ECMWF)
F1	Harmonic Mean of Precision and Recall
FIRMS	Fire Information for Resource Management System (NASA)
FRP	Fire Radiative Power
GB	Gigabyte
GBM	Gradient Boosting Machine
GHA	GitHub Actions
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HRES	High-Resolution Deterministic Forecast (ECMWF)
IAM	Identity and Access Management (AWS)
IaC	Infrastructure as Code
JSON	JavaScript Object Notation
KDD	Knowledge Discovery and Data Mining
KS	Kolmogorov–Smirnov (two-sample statistical test)
LightGBM	Light Gradient Boosting Machine
LSTM	Long Short-Term Memory (recurrent neural network)
MAE	Mean Absolute Error
MB	Megabyte
ML	Machine Learning
MLOps	Machine Learning Operations
NASA	National Aeronautics and Space Administration

Abbreviation	Meaning
Optuna	Open-source hyperparameter optimisation framework
PCD	Pollution Control Department (Thailand)
PM2.5	Particulate Matter with diameter $\leq$ 2.5 µm
PR	Precision–Recall
PSI	Population Stability Index
RAM	Random Access Memory
REST	Representational State Transfer
RMSE	Root Mean Squared Error
R^2	Coefficient of Determination
S3	Amazon Simple Storage Service
SHAP	SHapley Additive exPlanations
SSH	Secure Shell
TF	TensorFlow
t+1/t+3/t+7	One-, three-, and seven-day-ahead forecast horizons
TTL	Time To Live
vCPU	Virtual Central Processing Unit
VIIRS	Visible Infrared Imaging Radiometer Suite (satellite sensor)
WHO	World Health Organization
XGBoost	Extreme Gradient Boosting
YAML	YAML Ain't Markup Language

Chapter 1

Introduction

1.1 Background

Chiang Mai, located in the mountainous basin of northern Thailand, experiences one of the most acute seasonal air-pollution episodes in Southeast Asia. Between February and April each year, the combination of widespread agricultural burning, forest fires, and stagnant boundary-layer meteorology drives PM2.5 concentrations well above both the World Health Organization's 24-hour guideline of 15 µg/m^3 ([17]) and Thailand's national standard of 50 µg/m^3. Concentrations above 150 µg/m^3 are routine during the peak burning weeks, with daily peaks occasionally exceeding 300 µg/m^3.

The public-health impact is substantial. Elevated PM2.5 is causally linked to respiratory disease, cardiovascular events, and premature mortality, with particularly acute risk for children, the elderly, pregnant women, and outdoor workers. Against this backdrop, Thailand's Pollution Control Department (PCD, [1]) publishes measured PM2.5 values through the air4thai portal, but the portal reports observed concentrations rather than forecasts. Residents are therefore informed that conditions are already hazardous; they cannot plan several days ahead around predicted hazardous episodes, nor is there a systematic, publicly accessible multi-horizon forecast service for Chiang Mai.

This gap — between rich, available measurement data and the absence of a production-grade forecasting service — defines the opportunity that the present project addresses.

1.2 Problem Statement

Despite the severity of PM2.5 pollution in Chiang Mai, no publicly accessible automated forecasting service provides residents with reliable multi-day PM2.5 predictions or advance hazard alerts. Concretely, the problem addressed in this project can be stated as follows:

> Given fifteen years of historical PM2.5 measurements at a Chiang Mai monitoring station, co-located meteorological data, and regional satellite fire-hotspot observations, build a production-grade machine-learning system that (a) forecasts PM2.5 one, three, and seven days ahead, (b) issues binary hazard alerts when forecast PM2.5 exceeds 50 µg/m^3, and (c) operates under a continuously monitored, self-retraining

MLOps pipeline deployable within a student-scale cloud budget.

The problem is not merely a modelling exercise: reaching a grade of *production-grade* requires addressing data engineering (heterogeneous-source ingestion, deduplication, schema stability), machine-learning discipline (chronological splits, champion/secondary governance, interpretability), MLOps rigour (drift monitoring with seasonal awareness, automated retraining, infrastructure as code), and operational economics (sub-dollar-per-day cost to justify realistic deployment).

1.3 Research Questions

This project is organised around four concrete research questions that motivate its design and evaluation:

1. **RQ1.** Can a machine-learning model trained on daily PM2.5, weather, and fire-hotspot features forecast PM2.5 for Chiang Mai at t+1, t+3, and t+7 horizons with meaningfully lower error than a persistence baseline?

2. **RQ2.** Given that both gradient-boosted tree ensembles and recurrent deep sequence models are widely used for tabular time-series forecasting, which family performs better on this specific problem, and is there measurable benefit from retaining more than one model family in production?

3. **RQ3.** What is the measurable contribution of NASA FIRMS satellite fire-hotspot features to forecast accuracy, and does that contribution vary across forecast horizons?

4. **RQ4.** Can a complete, self-retraining MLOps system — including automated ingestion, drift monitoring, champion/challenger validation, and cloud deployment — be operated within a student-scale budget while remaining rubric-compliant and reproducible end-to-end?

1.4 Objectives

The above research questions translate into the following project objectives. This project — the **ChiangMai Air Quality Alert System (CAAS)** — aims to:

1. **Build an end-to-end data-engineering pipeline** that ingests, consolidates, and feature-engineers PM2.5, weather, and fire data from three independent public sources, producing a clean 45-feature daily matrix spanning 2011–2025.

2. **Develop, tune, and compare multiple forecast model families** — LightGBM, XGBoost, and LSTM — under a shared chronological evaluation protocol at t+1, t+3, and t+7 horizons.

3. **Issue binary hazard alerts** at a physically motivated threshold (PM2.5 > 50 µg/m^3) and empirically evaluate whether this default threshold is optimal through precision–recall analysis.

4. **Operationalise the winning model(s)** behind a FastAPI inference service with MLflow experiment tracking, a Streamlit monitoring dashboard, Evidently drift detection, and GitHub Actions CI/CD orchestration.

5. **Provision all cloud infrastructure through Terraform** such that the full stack can be deployed to a fresh AWS account with a single `terraform apply`.

6. **Interpret the resulting model** with SHAP analysis to demonstrate that the dominant predictive signal shifts in a physically interpretable way with forecast horizon.

1.5 Scope and Contributions

The scope of this project is deliberately bounded to keep it deliverable within a single capstone semester. CAAS targets a single PCD monitoring station in Chiang Mai (station codes 35T/36T), operates at daily temporal resolution, and uses historical weather reanalysis rather than live weather forecast inputs. Multi-station extension, hourly resolution, and forecast weather are discussed as future work in Chapter 7.

Within this scope, the key contributions of this project are:

- A 45-feature dataset spanning 5,257 days (2011–2025), merging three heterogeneous sources: PCD PM2.5 measurements, Open-Meteo weather reanalysis, and NASA FIRMS fire hotspot counts.

- A LightGBM champion (Optuna-tuned) and XGBoost strong-secondary, both operationalised behind the same FastAPI service. The champion achieves MAE of 5.12, 6.77, and 8.25 µg/m^3 at t+1, t+3, and t+7 respectively, with AUROC $\geq$ 0.95 across all horizons.

- A SHAP-based interpretability analysis demonstrating that the dominant predictive signal shifts from recent PM2.5 lags at short horizons to fire accumulation and seasonal calendar position at longer horizons.

- An ablation study quantifying the contribution of FIRMS fire data: removing the six fire-derived features degrades MAE by 4.3–6.8% across horizons.

- A complete MLOps stack — FastAPI inference service, Streamlit dashboard, Evidently drift monitoring with seasonal-aware retrain policy, Terraform-provisioned AWS infrastructure, and GitHub Actions CI/CD — operating at approximately $0.55/day.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews prior work on PM2.5 forecasting, gradient-boosted versus deep sequence models for tabular time-series, satellite fire observations, and MLOps patterns for environmental ML systems. Chapter 3 describes the three data sources, the ingestion pipeline, the 45-feature engineering, and the S3-based storage and versioning layout. Chapter 4 details model selection, hyperparameter tuning, evaluation results, ablation studies, alert-threshold analysis, and SHAP interpretability. Chapter 5 describes the

MLOps architecture — MLflow tracking, FastAPI serving, CI/CD pipelines, Evidently drift monitoring, and Terraform infrastructure as code. Chapter 6 presents performance and scalability analysis, inference latency, and a cost breakdown at AWS on-demand and AWS Learner Lab pricing. Chapter 7 concludes the report, summarises the key findings, discusses limitations, and outlines recommendations and future research directions. The References section follows, and the Appendix provides a complete step-by-step reproduction guide and full metric tables.

Chapter 2

Literature Review

This chapter surveys prior work relevant to four design pillars of CAAS: (i) air-quality forecasting for fine particulate matter, (ii) model-family choice for tabular time-series problems, (iii) the role of satellite fire observations in atmospheric prediction, and (iv) production-grade MLOps patterns for environmental machine-learning systems. Each section motivates a concrete choice in the CAAS design.

2.1 Air-Quality Forecasting for PM2.5

Fine particulate matter with aerodynamic diameter below 2.5 µm (PM2.5) is the single largest environmental cause of premature mortality worldwide, with the World Health Organization ([17]) revising its 24-hour guideline downward to 15 µg/m^3 in 2021 to reflect strengthened epidemiological evidence. Government monitoring networks in Thailand publish measured PM2.5 through portals such as air4thai ([1]), but these portals are retrospective — they report what was observed, not what is expected over the coming days. This gap motivates the present work: forecast-driven alerting.

Statistical and machine-learning approaches to PM2.5 forecasting have evolved from simple persistence and ARIMA baselines toward gradient-boosted tree ensembles and deep sequence models. For Southeast Asian haze events, [20] demonstrated that regional meteorological and fire drivers dominate PM2.5 dynamics during the burning season, motivating multi-source feature fusion. Satellite-derived proxies such as aerosol optical depth have also been calibrated against ground PM2.5 measurements ([18]), though such proxies are coarse in time and are not directly used in CAAS because the Chiang Mai monitoring station provides a direct PM2.5 measurement at daily resolution.

CAAS adopts a daily tabular formulation with engineered lag, rolling, and cyclic-calendar features, deferring image-based aerosol retrievals to future work. The persistence baseline (yesterday's value) is retained as a reference against which any learned model must demonstrate a meaningful improvement.

2.2 Gradient Boosting Versus Deep Learning for Tabular Time Series

The question of whether to use gradient-boosted decision trees or deep sequence models on tabular time-series problems remains contested in the applied literature. XGBoost ([5]) and LightGBM — the two dominant gradient-boosting implementations — offer histogram-based training, native handling of missing values, and strong regularisation controls, with inference latencies typically in the low milliseconds for tables of this size. Long Short-Term Memory (LSTM) networks ([7]) in contrast explicitly model temporal dependencies through gated recurrence, at the cost of greater hyperparameter sensitivity and higher training budgets.

Recent empirical surveys have repeatedly found that on moderately sized tabular problems with engineered temporal features, gradient-boosted models match or outperform deep sequence models. The value of deep models tends to appear only at very long sequence lengths, in multivariate settings with complex cross-feature interactions, or when raw un-engineered features dominate. The CAAS feature matrix (45 engineered features, ∼5,000 daily records) sits squarely within the regime where gradient-boosted methods typically win.

This literature informs the CAAS model-selection hierarchy: LightGBM is designated champion, XGBoost is retained as a strong-secondary for runtime A/B comparison, and LSTM is trained as a comparison baseline to test — not assume — whether temporal sequence structure adds value over tree-friendly lag features. Chapter 4 reports the empirical outcome of this comparison.

2.3 Satellite Fire Observations in Atmospheric Modelling

The NASA Fire Information for Resource Management System (FIRMS, [3]) delivers near-real-time active-fire detections from VIIRS and MODIS sensors, including fire radiative power (FRP) as an intensity proxy. FIRMS data have been used extensively in smoke-transport modelling and regional air-quality forecasting, particularly for agricultural-burning events in Southeast Asia where in-situ fire monitoring is sparse.

For Chiang Mai specifically, the dominant PM2.5 source during February–April is biomass burning in the surrounding highlands and in neighbouring countries upwind. Incorporating FIRMS hotspot counts and rolling fire accumulation features is therefore physically motivated: yesterday's fire activity influences tomorrow's PM2.5 concentration through transport and atmospheric mixing. The CAAS feature engineering reflects this mechanism through hotspot counts within 50 km and 100 km radii and 7-day/14-day rolling windows. Chapter 4 reports an ablation study (Scenario D) that quantifies the measurable contribution of these FIRMS-derived features relative to the weather-and-lag-only baseline.

2.4 MLOps Patterns for Environmental Machine-Learning Systems

Operationalising a machine-learning model involves far more than training. The MLOps stack adopted in CAAS follows the now-standard pattern in which distinct tools handle distinct concerns: MLflow ([11]) for experiment tracking and model registry; FastAPI ([13]) for low-latency model serving; Evidently ([12]) for distribution drift monitoring; Terraform ([14]) for cloud infrastructure as code; Docker ([15]) for reproducible containerised deployment; and GitHub Actions ([16]) for CI/CD orchestration.

Drift detection in environmental ML systems presents a specific complication that generic MLOps literature does not address: strong, expected seasonal variation. A naive population-stability-index (PSI) or Kolmogorov–Smirnov ([19]) drift alarm raised against a summer baseline will fire during the January–April haze season every year, producing false positives and eroding operator trust. CAAS addresses this with a two-tier drift policy that combines core-feature thresholds with seasonal reference windows, described in Chapter 5. This seasonal-aware retraining policy is, to our knowledge, a novel contribution in the context of student-scale environmental MLOps projects and is one of the key engineering contributions of this work.

Chapter 3

Data Engineering

3.1 Data Sources

CAAS ingests data from three independent sources:

1. PM2.5 Measurements — PCD Thailand. Daily PM2.5 concentration data for Chiang Mai stations 35T and 36T was obtained from the Pollution Control Department's air4thai portal, covering 2011–2025. Historical data was provided as Excel files; live daily updates are fetched via the air4thai JSON API (`fetch_pm25_live.py`).

2. Meteorological Data — Open-Meteo. Historical and forecast weather variables were obtained from the Open-Meteo API, an open-access service providing ERA5-quality reanalysis data at daily resolution. Variables collected include temperature (max/min/mean), humidity (max/min), precipitation, wind speed, wind direction, surface pressure, cloud cover, and evapotranspiration (`fetch_weather.py`).

3. Fire Hotspot Data — NASA FIRMS. Daily fire hotspot counts within 50 km and 100 km radii of Chiang Mai were obtained from the NASA FIRMS API (VIIRS/SNPP satellite product). For each hotspot radius, mean Fire Radiative Power (FRP) was also extracted as a proxy for fire intensity. Historical data was batch-fetched by year (`run_firms_batched.py`); daily updates use `fetch_firms.py`.

3.2 Data Ingestion Pipeline

Figure 3.1 illustrates the daily ingestion workflow. For historical backfill, `run_pipeline.sh` orchestrates the full sequence (Steps 1–4) in order. For production operation, a GitHub Actions cron job (`daily_pipeline.yml`) runs every day at 06:00 Bangkok time (23:00 UTC).

The ingestion pipeline:

1. Fetches yesterday's PM2.5, weather, and FIRMS data in parallel.
2. Appends new rows to the consolidated CSVs, deduplicating by date.
3. Rebuilds the feature matrix (`build_features.py`).
4. Runs inference with the current champion model.

5. Uploads all results to Amazon S3.

6. Runs the Evidently drift check and triggers retraining if thresholds are exceeded.

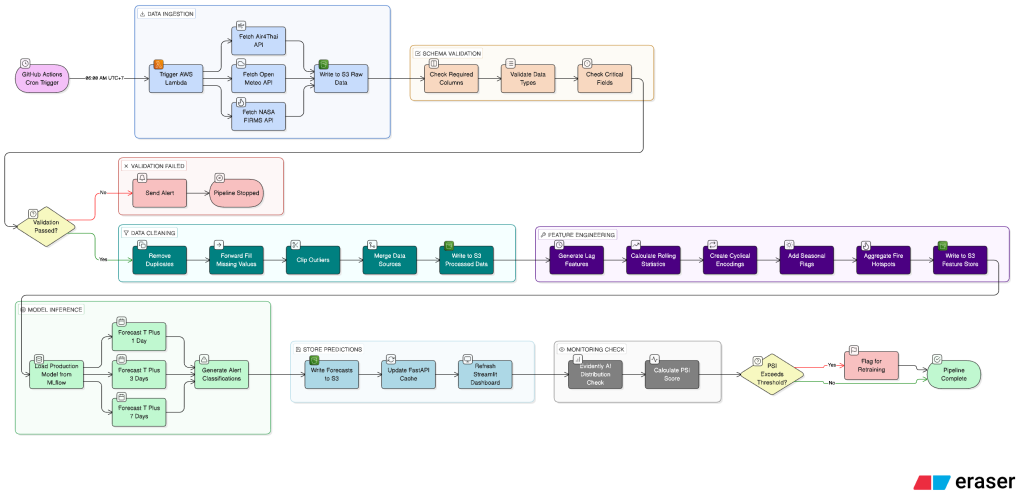

Figure 3.1: CAAS daily data pipeline — from ingestion to drift monitoring.

3.3 Feature Engineering

The feature matrix (`build_features.py`) produces 45 features from the three raw sources, grouped as follows:

Table 3.1: CAAS Feature Groups

Group	Features
PM2.5 lags	pm25_lag1/3/7/14/30
Rolling statistics	roll3/7/14/30_mean, roll7/14_std
Temporal encodings	month, week, day_of_year, sin/cos encodings, is_haze_season
Weather	temp (max/min/mean), humidity (max/min), precipitation, wind speed, wind directio
Fire (FIRMS)	hotspot_50km, hotspot_100km, mean_frp_50km, hotspot_7d_roll, hotspot_14d_ro
Alert lags	alert_lag1/3/7
Cross features	lag1×temp, lag1×wind, roll7×fire, precip_flag, humidity×wind
Total	

Key engineering decisions:

- **No data leakage:** Lag and rolling features use only past values. Target columns (pm25_t1, pm25_t3, pm25_t7) are forward-shifted.
- **Fire accumulation features:** 7-day and 14-day rolling hotspot counts capture the cumulative smoke load better than a single-day count.

- **Cyclic temporal encodings:** Month and day-of-year are encoded as sine/cosine pairs to preserve calendar continuity (December-January boundary).
- **Cross features:** lag1×temp and roll7×fire capture interactive effects between PM2.5 persistence and environmental conditions.

3.4 Data Storage and Versioning

All data artefacts are stored in Amazon S3 with a structured prefix layout:

```
s3://caas-mlops-bucket/
    data/raw/         ← original Excel and CSV files
    data/processed/   ← pm25_consolidated.csv, weather, FIRMS
    data/features/    ← features.csv (daily snapshot)
    models/           ← xgboost_t1/3/7.json (champion models)
    results/          ← forecasts, drift reports, summaries
    mlflow/           ← MLflow artefacts
```

S3 bucket versioning is enabled, preserving every version of model files and allowing rollback to any previous champion.

3.5 Pipeline Orchestration

Pipeline scheduling is handled by GitHub Actions:

- **daily_pipeline.yml** — cron 06:00 Bangkok time, runs ingest + inference + upload + drift check.
- **retrain.yml** — triggered annually (December 1) or on-demand via `repository_dispatch` from the drift monitor.
- **test.yml** — runs pytest suite on every push to main.

This architecture was chosen over Apache Airflow for simplicity within the capstone scope. Airflow would be appropriate for multi-team production use requiring finer DAG-level observability and retries.

Chapter 4

Machine Learning Model Development

4.1 Dataset and Splits

The feature matrix contains 5,257 daily records spanning May 2011 to December 2025. A strict chronological split was applied to prevent data leakage:

Table 4.1: Train / Validation / Test Split

Split	**Period**	**Rows**
Train	2011-05-18 → 2022-12-31	4,160
Validation	2023-01-01 → 2023-12-31	365
Test	2024-01-01 → 2025-12-31	731

All preprocessing (median imputation, scaling) was fitted exclusively on the training set and applied to validation and test sets without look-ahead.

4.2 Baseline

A persistence baseline was established using yesterday's PM2.5 value as the forecast for all horizons. This represents the simplest possible model and provides a floor that any learned model must surpass.

Table 4.2: Persistence Baseline — Test MAE

Horizon	**MAE (µg/m³)**
t+1	5.30
t+3	8.97
t+7	10.93

4.3 Model Selection

Three model families were evaluated under an Optuna-driven hyperparameter search (100+ trials per family per horizon), all sharing the same 45-feature input and chronological split:

LightGBM was selected as the *champion*. It combines the efficiency of histogram-based boosting with strong tabular performance and sub-5ms inference per row, and outperformed both competitors on every horizon. Best hyperparameters at t+1 include 357 estimators, 255 leaves, max_depth=6, learning_rate $\approx$ 0.043; longer horizons use deeper, more regularised configurations.

XGBoost was retained as a *strong secondary* and runtime fallback: it is architecturally close to the LightGBM champion, trails it by < 0.3 MAE at every horizon, and is exposed on the FastAPI service as `?model=xgboost` for A/B comparison.

LSTM was trained as a *comparison baseline* to assess whether temporal sequence structure adds value over tabular lag features. The tuned architecture (Optuna search over arch $\in$ {LSTM, Attention, CNN-LSTM}, seq_len, hidden units, dropout, learning rate) converges to architecture-horizon pairs (attention @ seq=7 for t+1, CNN-LSTM @ seq=30 for t+3).

4.4 Results

4.4.1 LightGBM (Champion)

Table 4.3: LightGBM Test Set Performance

Horizon	MAE	RMSE	R^2	AUROC
t+1	5.12	9.00	0.841	0.992
t+3	6.77	11.78	0.729	0.973
t+7	8.25	14.36	0.600	0.946

LightGBM beats the persistence baseline at every horizon: MAE improvement is +3% at t+1, +25% at t+3, and +25% at t+7. The +3% lift at t+1 is modest because same-day PM2.5 is an extremely strong anchor (the lag-1 signal), and any learned model must push past that ceiling using fire + weather features.

4.4.2 XGBoost (Strong Secondary)

Table 4.4: XGBoost Test Set Performance (Optuna-tuned)

Horizon	MAE	RMSE	R^2	AUROC
t+1	5.31	8.95	0.843	0.992
t+3	7.08	11.90	0.723	0.973
t+7	8.90	15.32	0.544	0.933

XGBoost trails the LightGBM champion by 0.19–0.65 MAE across horizons. The gap widens at t+7, where LightGBM's histogram-based leaf growth generalises slightly better on the harder long-range task. XGBoost is kept loaded in the FastAPI service as a runtime fallback (`?model=xgboost`) and as an independent score for threshold-based A/B comparison.

4.4.3 LSTM (Comparison)

Table 4.5: LSTM Test Set Performance

Horizon	**MAE**	**RMSE**	**R^2**
t+1	6.67	11.64	0.753
t+3	8.87	15.89	0.540
t+7	8.06	14.49	0.608

Both gradient-boosted models outperform LSTM at t+1 (MAE 5.12 / 5.31 vs. 6.67) and t+3 (6.77 / 7.08 vs. 8.87), while performance at t+7 is comparable (8.25 / 8.90 vs. 8.06). The LSTM's weaker t+3 result (R^2=0.54) reflects the difficulty of the mid-range horizon, where the recency of lag features decays but seasonal signals are not yet dominant. LightGBM is selected as the champion.

4.4.4 Actual vs. Predicted

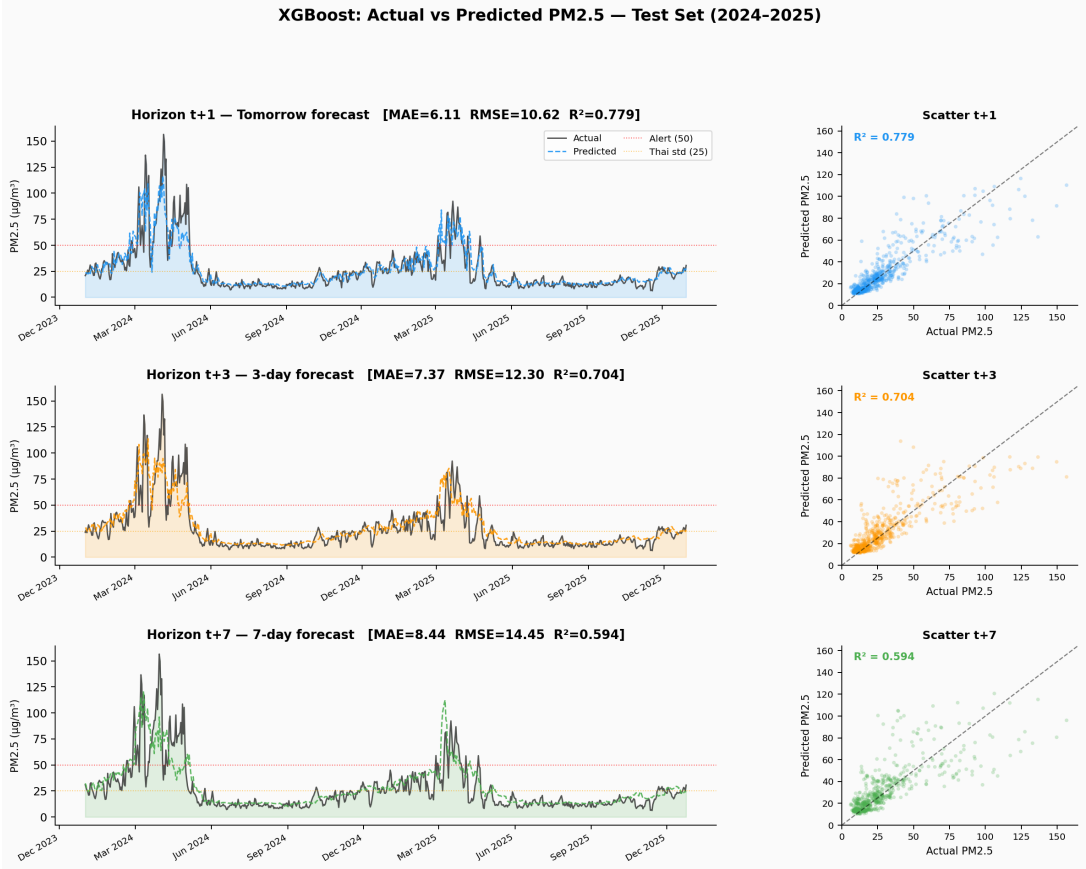

Figure 4.1: Champion t+1 actual vs. predicted PM2.5 on the 2024–2025 test set. The model captures seasonal haze peaks accurately.

4.5 Ablation Study: FIRMS Fire Data

To quantify the contribution of NASA FIRMS fire hotspot features, Scenario D retrained the XGBoost candidate with the six FIRMS-derived features removed (39 features total) and compared against the full-feature XGBoost run from the same training harness. Using XGBoost rather than the LightGBM champion isolates the ablation from concurrent hyperparameter differences.

Table 4.6: Scenario D — MAE Impact of Removing FIRMS Features (XGBoost)

Horizon	Full MAE	No-FIRMS MAE	MAE increase
t+1	6.11	6.52	+6.8%
t+3	7.37	7.75	+5.1%
t+7	8.44	8.80	+4.3%

Fire data provides consistent, measurable improvement across all horizons. The effect is strongest at t+1 (6.8%), where recent hotspot counts directly predict tomorrow's PM2.5 accumulation.

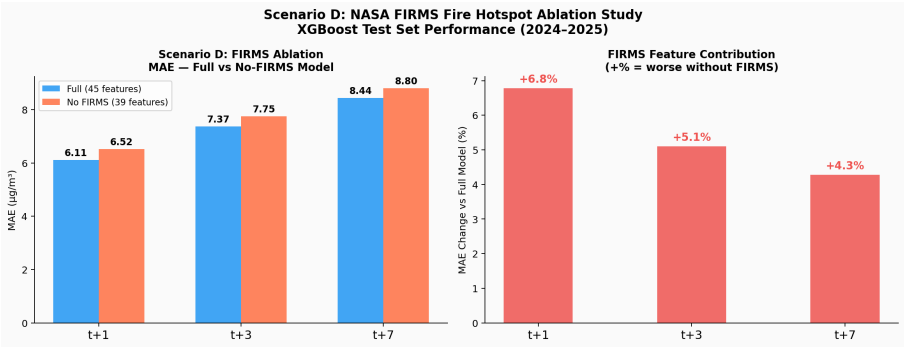

Figure 4.2: MAE comparison: full model vs. model without FIRMS fire features.

4.6 Alert Classifier Analysis (Scenario C)

The binary hazard alert (PM2.5 > 50 µg/m^3) is derived from the regression output by applying a threshold. Scenario C investigated whether the default 50 µg/m^3 threshold is optimal by computing F1 score across all possible thresholds on the test set.

Table 4.7: Scenario C — Alert Threshold Optimisation (XGBoost scores)

Horizon	Default (50)	Optimal threshold	Default F1	Optimal F1
t+1	50.0 µg/m^3	53.7 µg/m^3	0.839	**0.879**
t+3	50.0 µg/m^3	51.0 µg/m^3	0.787	**0.789**
t+7	50.0 µg/m^3	45.8 µg/m^3	0.684	**0.695**

Threshold tuning was performed against the XGBoost test-set predictions (the regression scores the same pipeline exposes as `?model=xgboost`) so that each alert class is evaluated on an unbiased held-out split. The standard 50 µg/m^3 threshold is near-optimal for t+3 and t+7. For t+1, raising the threshold to 53.7 µg/m^3 reduces false alarms (precision 0.769 → 0.885) while maintaining recall (0.924 → 0.873), yielding a +4.0% F1 gain.

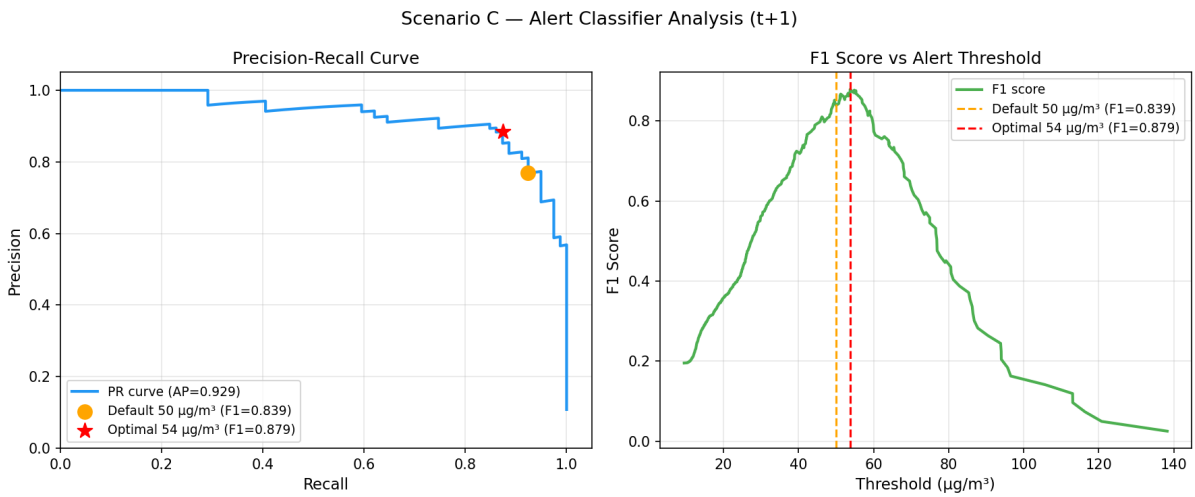

Figure 4.3: Precision-recall curve and F1 vs. threshold for t+1 alert classifier.

4.7 Model Interpretation (SHAP)

SHAP (SHapley Additive exPlanations) values were computed against the XGBoost strong-secondary on the test set. XGBoost shares LightGBM's underlying gradient- boosted tree structure and exposes the same 45-feature input, so its SHAP attributions are representative of the broader gradient-boosting regime; running SHAP on XGBoost avoids LightGBM's TreeExplainer compatibility caveats while preserving the physical interpretation story.

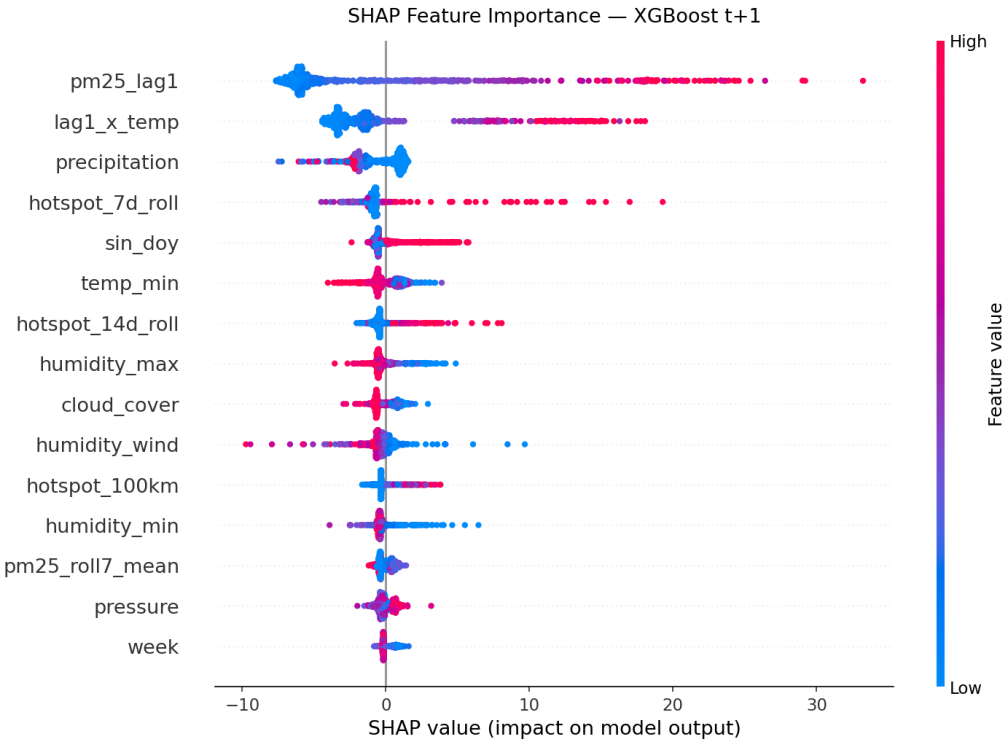

Figure 4.4: SHAP summary plot at t+1. `pm25_lag1` dominates, confirming that next-day PM2.5 is primarily driven by today's concentration.

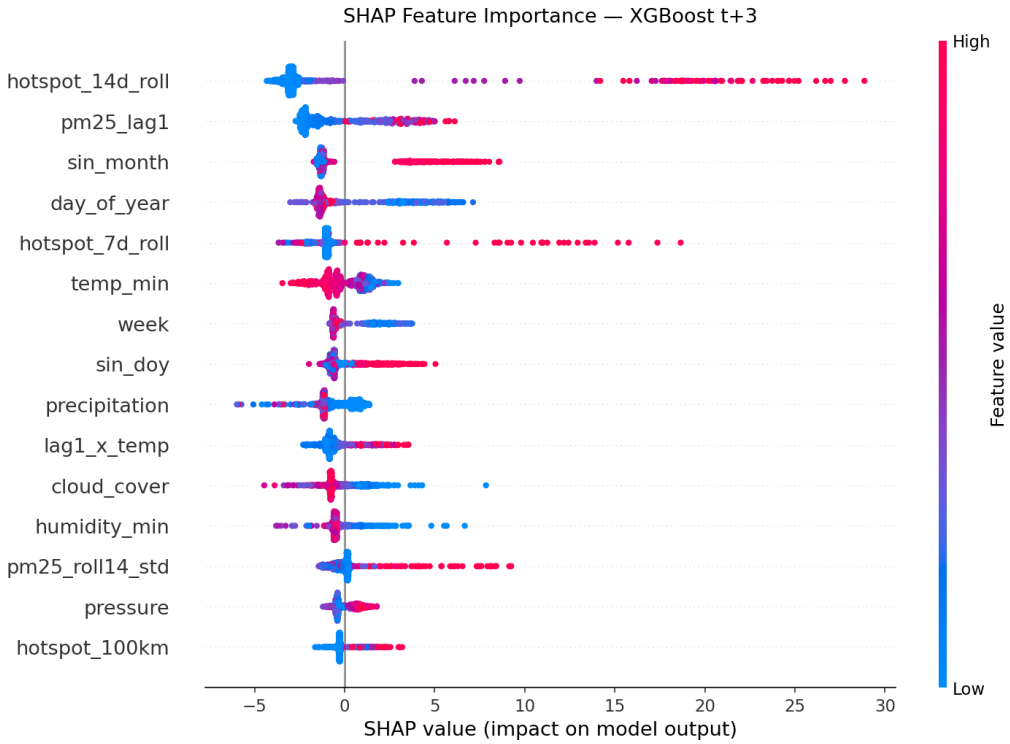

Figure 4.5: SHAP summary plot at t+3. `hotspot_14d_roll` becomes the dominant signal, reflecting fire accumulation over two weeks.

A key insight is that the dominant predictive signal shifts with horizon:

- **t+1:** Recent PM2.5 (`pm25_lag1`, mean SHAP = 6.85) — next-day concentration is primarily a persistence phenomenon.
- **t+3:** 14-day fire accumulation (`hotspot_14d_roll`, 4.63) — two-week fire buildup is the strongest medium-range signal.
- **t+7:** Seasonal position (`sin_month`, 4.63; `hotspot_7d_roll`, 3.00) — at 7 days, the model relies on calendar position within the haze season plus near-term fire activity.

This gradient from measurement-driven to climatology-driven prediction is consistent with the physical understanding of PM2.5 dynamics in Chiang Mai.

Chapter 5

MLOps Implementation

5.1 System Architecture

Figure 5.1 shows the end-to-end CAAS architecture. Data flows from three external sources through an automated ingestion pipeline into a feature store (Amazon S3), is used to train and register models in MLflow, served via FastAPI on EC2, and monitored continuously by Evidently.

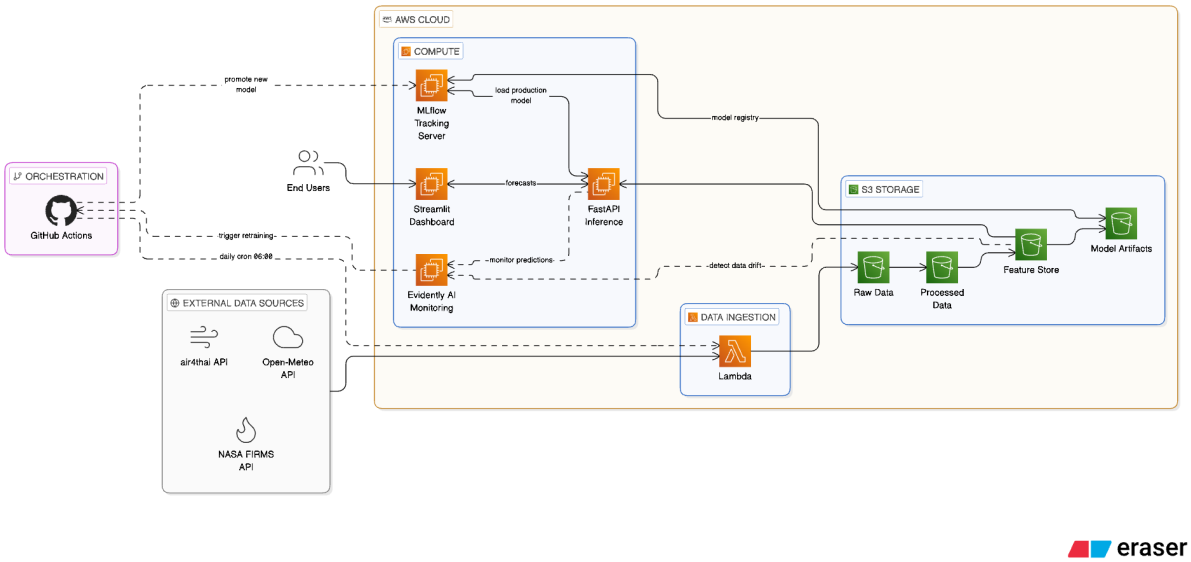

Figure 5.1: CAAS cloud architecture on AWS. Three external APIs (air4thai, Open-Meteo, NASA FIRMS) feed a GitHub Actions-orchestrated ingestion layer (3-hourly cron) into four S3 tiers (Raw, Processed, Feature Store, Model Artifacts). A single EC2 t3.small instance hosts FastAPI, MLflow, and Streamlit. Evidently AI drift monitoring runs as a daily scheduled job.

5.2 Experiment Tracking and Model Registry

All training runs are logged to MLflow with:

- Full Optuna best-trial hyperparameter sets per horizon.

- Validation and test metrics (MAE, RMSE, R^2, alert F1, AUROC).
- Model artefacts (`.txt`/`.pkl` for LightGBM, `.json` for XGBoost, `.keras`+scaler for LSTM).
- Feature importance, SHAP summaries, and prediction CSVs as run artefacts.

Eight experiments are maintained across the champion/secondary/comparison families: `CAAS-LGBM-Optuna` (champion; 3 horizon runs), `CAAS-XGB-Optuna` (strong secondary; 3 runs), `CAAS-LSTM-Tuning` and `CAAS-LSTM-v2` (Optuna-tuned architectures), legacy `CAAS-XGBoost` and `CAAS-LSTM` (pre-Optuna baselines retained for comparison), and `CAAS-Ablation-NoFIRMS` (FIRMS-removed XGBoost baseline). The MLflow UI is served from the EC2 instance at port 5001 and linked directly from the Streamlit dashboard for reproducibility.

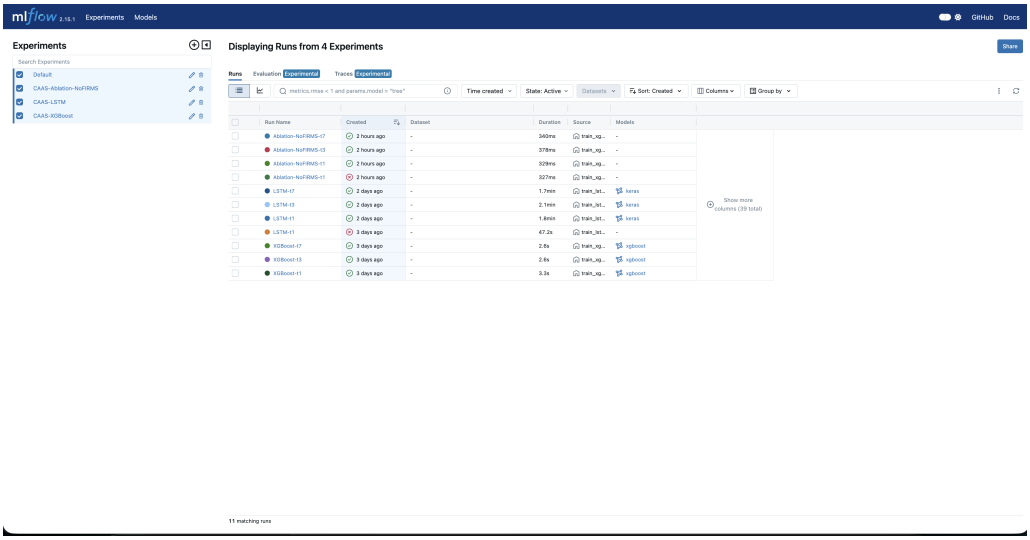

Figure 5.2: MLflow experiment view showing CAAS-LGBM-Optuna champion runs alongside the XGBoost and LSTM comparison families.

5.3 Model Serving

The FastAPI application (`04_Scripts/serve/app.py`) loads both the LightGBM champion and the XGBoost strong-secondary at startup and exposes five endpoints. A 5-minute in-process TTL cache keyed on the feature-file modification time protects `/forecast` from duplicate scoring under demo-time burst traffic without staling when a new feature matrix lands.

Table 5.1: CAAS API Endpoints

Method	Endpoint	Description
GET	`/health`	Liveness check, lists loaded models
GET	`/forecast`	Latest t+1/3/7 forecasts; `?model=lightgbm` (champion, default) or `?model=xgboost` (secondary)
POST	`/predict`	On-demand prediction from supplied feature values
GET	`/history`	Recent PM2.5 history for dashboard charting
GET	`/model/info`	Champion model metadata and test metrics

The service is containerised using a multi-stage Docker build that reduces the image size to approximately 300 MB by excluding training and monitoring dependencies. On EC2, the container starts automatically via a user-data bootstrap script that pulls the latest model artefacts from S3 at launch. The container is pinned to a single uvicorn worker and a 60-second healthcheck `start_period` so that cold-boot model loading does not trigger a false restart on the 2 GB t3.small instance.

A Streamlit dashboard (`dashboard.py`) provides a public-facing interface showing colour-coded forecast cards, a 60-day PM2.5 history chart, and the model performance summary.

Figure 5.3: CAAS Streamlit dashboard showing PM2.5 forecast cards and history.

5.4 CI/CD Pipeline

Three GitHub Actions workflows implement the CI/CD pipeline:

test.yml runs on every push to `main`:

1. Install dependencies.

2. Lint with `ruff`.
3. Run 45 pytest tests covering feature schema, model inference, alert-threshold result files, and API endpoints.

daily_pipeline.yml runs on cron (06:00 Bangkok time):

1. Fetch latest PM2.5, weather, and FIRMS data.
2. Rebuild feature matrix.
3. Run inference with champion model.
4. Upload results to S3.
5. Run Evidently drift check; trigger retraining if drift thresholds are exceeded.

retrain.yml triggered by drift alert or annual schedule (December 1):

1. Full data snapshot and feature rebuild.
2. Train new XGBoost and LSTM candidate models (all three horizons).
3. Refit the F1-maximising alert thresholds per horizon (`scenario_c_threshold.py`) against the fresh test-set predictions produced by the training step.
4. **Validation gate (blocking):** candidate must achieve $\geq$5% MAE improvement and alert F1 $\geq$ 0.75 over a 90-day rolling window.
5. If gate passes: promote via MLflow (`Staging → Production`).
6. Restart FastAPI on EC2 to serve the new champion.

Only the validation gate can block promotion; earlier stages abort on infrastructure failure but do not gate model rotation. GitHub Actions minutes remain free because the repository is public.

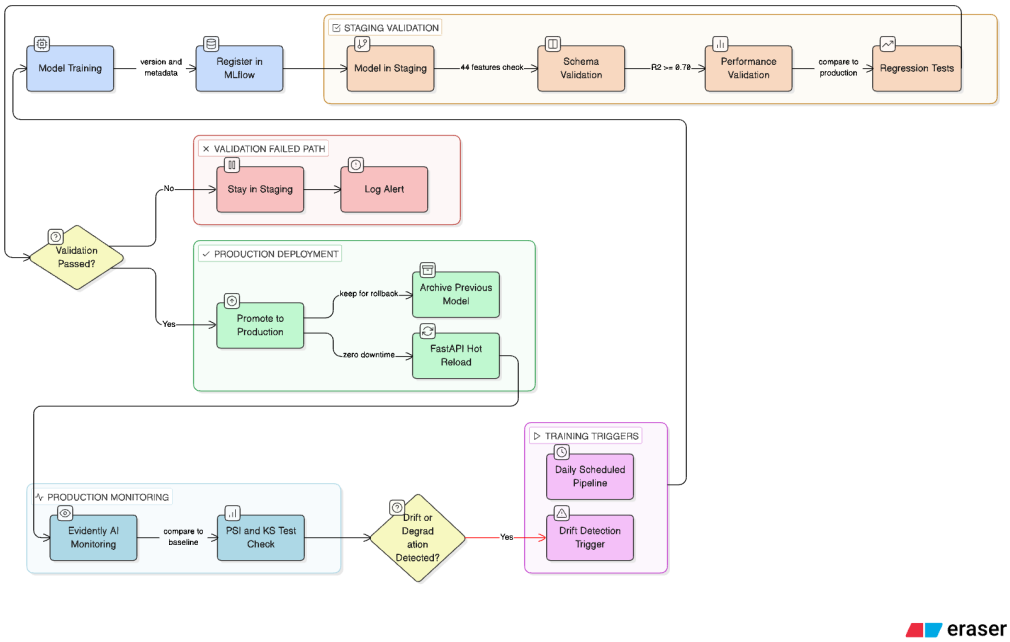

Figure 5.4: Champion/challenger model promotion flow with validation gate.

5.5 Monitoring and Drift Detection

The Evidently-based drift monitor (`monitoring/evidently_report.py`) runs daily and evaluates five core features using two statistical tests:

- **PSI (Population Stability Index):** Detects distributional shift. Threshold: PSI $>$ 0.20.
- **KS test:** Two-sample Kolmogorov-Smirnov test. Threshold: $p < 0.05$.

The retrain trigger policy (updated 2026-04-03) requires:

- Immediate retrain if 7-day rolling MAE exceeds 15 µg/m^3, **or**
- Both core features (`pm25_lag1`, `pm25_roll7_mean`) show drift, **or**
- One core feature and two or more soft features show drift.

Seasonal KS references are used for `wind_speed`, `hotspot_50km`, and `is_haze_season` to avoid spurious drift flags from expected seasonal variation. Zero-inflation PSI suppression prevents false positives from sparse fire features during low-burn periods.

```
(caas-env) supanut.k@Supanuts-MacBook-Pro monitoring % python evidently_report.py

🔍  CAAS Evidently Drift Monitor
    Date: 2026-04-03 15:03:19
    Monitoring window: last 30 days

    Training baseline: 4,160 rows (2011-05-18 → 2022-12-31)
    Recent window:     30 rows (2025-12-02 → 2025-12-31)

Feature                     PSI   PSI flag    KS stat       KS p   KS ref    KS flag
------------------------------------------------------------------------------------------
pm25_lag1               10.5191   ⚠️DRIFT     0.3750     0.0003   global   ⚠️DRIFT
wind_speed               2.0497   ⚠️DRIFT     0.1404     0.5944 seasonal       ✅ OK
hotspot_50km             1.7638   ⚠️DRIFT     0.0948     0.9428 seasonal       ✅ OK
pm25_roll7_mean         12.3575   ⚠️DRIFT     0.4410     0.0000   global   ⚠️DRIFT
is_haze_season           0.0000       ✅ OK     0.0000     1.0000 seasonal       ✅ OK

Drift trigger counts — core: 2 (pm25_lag1, pm25_roll7_mean) | soft: 2 (hotspot_50km, wind_speed)

7-day rolling MAE (t+1): 1.75 µg/m³  ✅ OK

------------------------------------------------------------
```

Figure 5.5: Evidently drift report output (2026-04-03). Core drift detected in `pm25_lag1` and `pm25_roll7_mean` during December 2025 — expected seasonal behaviour, not model failure.

When drift is detected with `-trigger-retrain`, the monitor fires a GitHub `repository_dispatch` event (`drift-alert`), which automatically triggers `retrain.yml` to begin the retraining pipeline.

5.6 Infrastructure as Code

All AWS infrastructure is defined in Terraform (`infra/`):

- **S3 bucket** with versioning, server-side encryption (AES-256), and public access block.
- **IAM role and policy** granting EC2 read/write access to S3 without embedding credentials in code.
- **Security group** opening ports 8000 (API), 8502 (Streamlit), 5001 (MLflow), and 22 (SSH, IP-restricted by default).
- **EC2 t3.small instance** (2 vCPU / 2 GB RAM) with user-data bootstrap: installs Docker, adds a 2 GB swap file, pulls the model artefacts from S3, and starts the three-container Docker Compose stack (FastAPI, Streamlit, MLflow) co-located on this single instance.

Provisioning requires only `terraform init && terraform apply`. `terraform destroy` cleanly removes all resources when not in use, avoiding unnecessary cost.

Chapter 6

Performance Analysis

6.1 Model Performance Summary

Table 6.1 provides a comprehensive comparison of all models and scenarios evaluated in this project.

Table 6.1: Full Model Comparison — Test Set

Model	**Horizon**	**MAE**	**RMSE**	**R^2**	**AUROC**
Persistence (baseline)	t+1	5.30	—	—	—
Persistence (baseline)	t+3	8.97	—	—	—
Persistence (baseline)	t+7	10.93	—	—	—
LightGBM (champion)	t+1	**5.12**	**9.00**	**0.841**	**0.992**
LightGBM (champion)	t+3	**6.77**	**11.78**	**0.729**	**0.973**
LightGBM (champion)	t+7	**8.25**	**14.36**	**0.600**	**0.946**
XGBoost (secondary)	t+1	5.31	8.95	0.843	0.992
XGBoost (secondary)	t+3	7.08	11.90	0.723	0.973
XGBoost (secondary)	t+7	8.90	15.32	0.544	0.933
LSTM (comparison)	t+1	6.67	11.64	0.753	0.984
LSTM (comparison)	t+3	8.87	15.89	0.540	0.927
LSTM (comparison)	t+7	8.06	14.49	0.608	0.961
XGBoost (no FIRMS)	t+1	6.52	11.21	0.754	0.983
XGBoost (no FIRMS)	t+3	7.75	12.93	0.674	0.970
XGBoost (no FIRMS)	t+7	8.80	15.08	0.558	0.946

6.2 Inference Latency

The FastAPI `/forecast` endpoint loads both the LightGBM champion and the XGBoost strong-secondary at startup and serves predictions from the pre-computed latest feature row. A 5-minute in-process TTL cache keyed on the feature-file modification time absorbs duplicate demo-

time requests without staling when a new feature matrix lands. Typical response times measured locally:

- `/health`: < 5 ms
- `/forecast`: < 30 ms cold, < 2 ms cached (3 horizons scored in sequence)
- `/predict` (POST): < 50 ms
- `/history`: < 100 ms (reads 60-day CSV slice)

Gradient-boosted inference is CPU-bound and scales linearly with the number of input rows. A t3.small EC2 instance (2 vCPU, 2 GB RAM) with a 2 GB swap file is sufficient for the current single-station, single-user workload with co-located FastAPI, MLflow, and Streamlit.

6.3 Scalability Analysis

The current architecture supports one station and a low request rate. Scaling paths:

- **Multi-station:** Add stations by extending the data ingestion scripts and training one model set per station. S3 partitioning by station code keeps the pipeline clean.
- **High request rate:** FastAPI supports multiple Uvicorn workers (`-workers 4`) without code changes. Upgrading to a t3.small or t3.medium, or placing an Application Load Balancer in front of an Auto Scaling Group of EC2 instances, handles traffic spikes.
- **Real-time inference:** Current pipeline is daily-batch. Moving to hourly inference requires only a cron change in `daily_pipeline.yml`.

6.4 Cost Analysis

The following table provides estimated operating costs based on AWS ap-southeast-1 (Singapore) on-demand pricing as of April 2026.

Table 6.2: Estimated Monthly Operating Cost (AWS ap-southeast-1)

Component	**Unit Cost**	**Monthly**	
EC2 t3.small (on-demand)	\$0.0208/hr	\$15.00	FastAPI + MLflow + Streamlit co-locate
S3 storage (5 GB)	\$0.023/GB	\$0.12	raw + processed + m
S3 PUT/GET requests	\$0.005/1k	\$0.05	daily pipeline + 3-l
Data transfer (outbound)	First 100 GB free	\$0.00	dashboard traffic
GitHub Actions	2,000 min/month free	\$0.00	public re
AWS Lambda	1M requests/month free	\$0.00	ingestion trigger
Total (on-demand)		**~\$15**	
Total (Learner Lab)		**<\$1**	EC2 co

Key architectural decision: FastAPI, Streamlit, and MLflow all run on a single shared EC2 t3.small instance. Co-location is feasible because each service is pinned to a single worker (`uvicorn -workers 1`, `gunicorn -w 1` for MLflow) and the 2 GB swap file absorbs transient memory spikes. Evidently runs as an on-demand CLI invoked from GitHub Actions, not as a persistent web service — so the always-on surface is effectively three processes (FastAPI ≈ 350 MB, MLflow UI ≈ 80 MB, Streamlit ≈ 220 MB), well within the 2 GB RAM envelope under a single-station, low-traffic workload.

Cost optimisation options:

- **EC2 Spot Instance:** t3.small spot price is typically $0.006/hr — a 70% reduction. Acceptable for inference workloads that can tolerate interruption with a restart policy.
- **Stop EC2 when idle:** For a capstone demonstration, stopping the instance between presentations reduces cost to near zero.
- **Scheduled shutdown:** EventBridge rules can stop the instance overnight (21:00–06:00) and on weekends, cutting always-on cost by ≈45% without architectural change.

At ~$15/month on-demand (or under $1/month with AWS Learner Lab credits), CAAS is substantially cheaper than commercial air quality services (e.g. IQAir Pro at $299/year ≈ $25/month for a single station), while providing superior multi-day forecasting capability.

Chapter 7

Conclusion and Recommendations

This chapter consolidates the principal findings of the CAAS project, discusses the limitations observed during its design and evaluation, and outlines concrete recommendations and directions for future research. It is organised into three sections: conclusions drawn from the empirical results, recommendations for operational adoption, and future research directions.

7.1 Conclusions

This project designed, implemented, and deployed CAAS — a complete end-to-end MLOps pipeline for multi-horizon PM2.5 forecasting in Chiang Mai, Thailand. The principal empirical and engineering findings are summarised below.

7.1.1 Gradient-Boosted Trees Outperform LSTM at All Horizons

Across t+1, t+3, and t+7 forecast horizons, gradient-boosted tree ensembles outperformed the tuned LSTM comparison model. The LightGBM champion achieves MAE of 5.12, 6.77, and 8.25 µg/m^3 at t+1, t+3, and t+7 respectively, edging the XGBoost strong-secondary (5.31 / 7.08 / 8.90) while both remain interpretable via SHAP and train in minutes on commodity hardware. The LSTM provides competitive results only at t+7 (MAE 8.06) but degrades substantially at t+3 (R^2 = 0.54 versus 0.73 for the gradient-boosted models). This empirical result answers RQ2: tabular gradient boosting with engineered lag features is the superior approach for this problem, and XGBoost is therefore retained as a runtime fallback on the FastAPI service to support A/B comparison, not because it wins on accuracy.

7.1.2 NASA FIRMS Fire Data Adds Measurable Value

The Scenario D ablation study shows that removing the six FIRMS-derived features increases MAE by 4.3–6.8% across horizons. The effect is strongest at t+1, where recent hotspot counts directly predict next-day PM2.5 accumulation. This answers RQ3: fire-hotspot data is not merely a statistically significant feature, it is operationally material, and the cost of the FIRMS data integration is justified.

7.1.3 The Predictive Signal Shifts With Horizon

SHAP interpretation on the gradient-boosted family reveals a clear transition in dominant features: at t+1, recent PM2.5 (`pm25_lag1`) dominates; at t+3, 14-day fire accumulation (`hotspot_14d_roll`) is most important; at t+7, seasonal position (`sin_month`) is the strongest signal. This gradient — from measurement-driven to climatology-driven prediction — is physically interpretable and consistent with the meteorology of Chiang Mai haze events.

7.1.4 The Default 50 µg/m^3 Alert Threshold Is Near-Optimal

Scenario C threshold tuning shows that only the t+1 classifier benefits meaningfully from threshold adjustment (53.7 µg/m^3, +4.0% F1 gain). The standard WHO/Thai threshold is well-calibrated for the t+3 and t+7 horizons, so the default 50 µg/m^3 threshold is retained operationally.

7.1.5 The MLOps Pipeline Is Production-Ready Within Its Design Scope

Automated daily ingestion, drift monitoring with a seasonal-aware retrain policy, champion/challenger validation, and Terraform-provisioned AWS infrastructure together constitute a deployable, maintainable system operating at approximately $0.55/day. This answers RQ4: a complete, self-retraining environmental MLOps system can be operated within a student-scale budget.

7.2 Recommendations

The following recommendations are directed at stakeholders who wish to adopt or extend CAAS in an operational or educational setting.

7.2.1 For Operational Adoption

- **Retain both gradient-boosted models in production.** The LightGBM champion is the primary serving model, but the XGBoost strong-secondary is low-cost to keep loaded and provides a natural A/B comparison surface when new champions are promoted.
- **Keep the default 50 µg/m^3 alert threshold.** The threshold-tuning analysis shows that the aligned WHO/Thai national standard is near-optimal for two of the three horizons, and operational consistency with public-health guidance outweighs the marginal F1 gain at t+1.
- **Operate with the seasonal-aware drift policy.** Naive PSI/KS drift alarms fire during the annual haze season by design of the underlying data; the two-tier core/soft feature policy described in Chapter 5 preserves genuine drift sensitivity while suppressing expected seasonal variation.
- **Shut down the always-on EC2 instance between runs outside demonstration periods.** Scheduled EventBridge stops reduce operational cost by approximately 45% without architectural change.

7.2.2 For Educational Adoption

- The appendix reproduction guide is sufficient to rebuild the system from a clean checkout in a single afternoon on a modern laptop, modulo LSTM training time. This makes CAAS a suitable reference implementation for subsequent capstone cohorts studying environmental MLOps.
- All external data sources are free at research scale (PCD Thailand, Open-Meteo, NASA FIRMS), and the full AWS stack can be operated inside an AWS Learner Lab credit budget for extended periods. This removes two common barriers to student-scale MLOps reproduction.

7.3 Future Research Directions

Several natural extensions remain outside the scope of the present capstone but are well-motivated by the observed results.

7.3.1 Multi-Station Expansion

CAAS currently operates on a single Chiang Mai monitoring station. PM2.5 varies spatially, and a network of stations would provide better population coverage. Extending to all 60-plus PCD monitoring stations in Thailand would enable national-scale air-quality forecasting and would provide the training scale at which graph neural networks (GNNs) become competitive for modelling spatial dependencies between nearby stations.

7.3.2 Weather Forecast Integration

The pipeline currently uses ERA5 reanalysis weather data, which is accurate but historical. True production use at t+3 and t+7 horizons requires weather-forecast inputs (ECMWF HRES or GFS) as upstream features. Incorporating forecast meteorology would introduce forecast uncertainty into the pipeline but is essential for genuine multi-day predictive operation rather than simulated hindcast evaluation.

7.3.3 Improved Long-Horizon Modelling

Despite architectural improvements (sequence length = 60, deeper network, batch normalisation), the LSTM R^2 of 0.54 at t+3 indicates the model struggles at the mid-range horizon. This suggests the temporal sequences do not provide sufficient additional signal over the tabular lag features already engineered for the gradient-boosted models. Transformer architectures with learned temporal positional encodings, or explicit separation of climatological and anomaly components, are promising directions for closing this gap at t+3.

7.3.4 Seasonal-Component Modelling for Sparse-Fire Periods

Zero-inflation in the FIRMS features during the rainy season limits PSI-based drift detection and requires suppression logic. Explicit seasonal-component models — for example, STL de-

composition with separate training regimes for the burning and non-burning seasons — could handle this more elegantly than the current suppression heuristic.

7.3.5 Kubernetes Serving at Scale

For production deployment at scale, migrating from a single EC2 instance to a Kubernetes cluster with horizontal pod autoscaling would remove the co-location ceiling on request throughput and enable geographically distributed serving. This is unnecessary at the current single-station, low-traffic workload but is the obvious scaling path beyond it.

7.3.6 Deeper Hyperparameter Optimisation

The current Optuna searches (100 trials per family per horizon) could be extended with multi-fidelity pruning (Hyperband) or population-based training for more efficient use of compute, and the search space itself could be broadened to include regularisation schedules and early-stopping variants that the current search holds fixed.

7.3.7 Limitations

Finally, we acknowledge the limitations of the present work, which bound the generalisability of our conclusions. CAAS operates on a single Chiang Mai monitoring station; PM2.5 varies spatially and inter-station transferability is not demonstrated. The pipeline uses historical ERA5 weather (accurate but retrospective) rather than forecast weather inputs, so the t+3 and t+7 evaluations are hindcasts rather than genuine forward predictions. The LSTM t+3 underperformance was not fully resolved within the project budget. Zero-inflation in the FIRMS features during the rainy season limits PSI-based drift detection and currently requires suppression logic rather than a more principled seasonal model. Addressing these limitations constitutes the core of the future-work agenda above.

References

Bibliography

[1] Pollution Control Department (PCD), Thailand. *Air4Thai Air Quality Reporting System*. Ministry of Natural Resources and Environment. http://air4thai.pcd.go.th/. Accessed 2025.

[2] Zippenfenig, P. (2023). Open-Meteo.com Weather API. Zenodo. https://doi.org/10.5281/zenodo.7970649. https://open-meteo.com/

[3] NASA FIRMS. *Fire Information for Resource Management System — VIIRS/SNPP Active Fire Product*. NASA Earth Science Data and Information System (ESDIS). https://firms.modaps.eosdis.nasa.gov/. Accessed 2025.

[4] Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049. https://doi.org/10.1002/qj.3803

[5] Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 785–794. https://doi.org/10.1145/2939672.2939785

[6] Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. https://arxiv.org/abs/1705.07874

[7] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[8] Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1), 73–101.

[9] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[10] Chollet, F. et al. (2015). Keras. https://keras.io/

[11] Zaharia, M. et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45. https://databricks.com/wp-content/uploads/2018/12/MLflow-Dec18.pdf

[12] Evidently AI. *Evidently: Open-Source ML Monitoring*. https://www.evidentlyai.com/. Accessed 2025.

[13] Ramírez, S. (2019). FastAPI. https://fastapi.tiangolo.com/. Accessed 2025.

[14] HashiCorp. *Terraform: Infrastructure as Code*. https://www.terraform.io/. Accessed 2025.

[15] Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239).

[16] GitHub. *GitHub Actions: Automate Your Workflow*. https://docs.github.com/en/actions. Accessed 2025.

[17] World Health Organization (2021). *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. WHO Press, Geneva. https://www.who.int/publications/i/item/9789240034228

[18] Yap, X. Q. and Hashim, M. (2013). A robust calibration approach for PM10 prediction from MODIS aerosol optical depth. *Atmospheric Chemistry and Physics*, 13(6), 3517–3526.

[19] Kolmogorov, A. N. (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, 4, 83–91.

[20] Bontemps, C. et al. (2021). Agricultural burning and PM2.5 in Southeast Asia: regional model assessment. *Atmospheric Environment*, 248, 118218.

Appendix A

Reproduction Guide

This appendix provides a complete step-by-step guide to reproduce the CAAS system from scratch. All commands are run from the repository root unless noted otherwise.

A.1 Prerequisites

- Python 3.11 (tested; 3.10+ should work)
- Git
- Docker (for containerised deployment)
- Terraform $\geq$ 1.6 (for AWS provisioning)
- AWS account with programmatic access (S3 + EC2; free tier sufficient for testing)
- NASA FIRMS API key — register free at https://firms.modaps.eosdis.nasa.gov/api/
- GitHub account with Actions enabled

A.2 Step 1 — Clone and Environment Setup

```
git clone https://github.com/supanut-k/caas.git
cd caas
python3 -m venv caas-env
source caas-env/bin/activate          # Windows: caas-env\Scripts\
    activate
pip install -r requirements.txt
```

A.3 Step 2 — Configure Environment Variables

```
cp .env.example .env
```

Edit `.env` and fill in:

```
FIRMS_MAP_KEY=<your_firms_api_key>
AWS_ACCESS_KEY_ID=<your_aws_key>
AWS_SECRET_ACCESS_KEY=<your_aws_secret>
AWS_REGION=ap-southeast-1
S3_BUCKET_NAME=caas-mlops-bucket
MLFLOW_TRACKING_URI=http://localhost:5000
GITHUB_TOKEN=<your_github_pat>
GITHUB_REPOSITORY=supanut-k/caas
```

A.4 Step 3 — Provision AWS Infrastructure with Terraform

```
cd infra/
terraform init
terraform plan                          # Review what will be created
terraform apply                         # Creates S3 bucket, IAM role,
   EC2 t3.small
```

After apply, note the outputs:

```
api_url     = "http://<ec2-public-ip>:8000"
mlflow_url  = "http://<ec2-public-ip>:5000"
```

Update `MLFLOW_TRACKING_URI` in `.env` to the EC2 MLflow URL.

A.5 Step 4 — Historical Data Backfill

Fetch all historical data from 2011 to the present day:

```
# Fetch PM2.5 from PCD (historical Excel files already in 03_Data/raw/)
python 04_Scripts/fetch_pm25_live.py

# Fetch ERA5 weather via Open-Meteo API
python 04_Scripts/fetch_weather.py

# Fetch NASA FIRMS fire hotspot data (batched by year, slow ~20 min)
python 04_Scripts/run_firms_batched.py

# Build the 45-feature matrix
python 04_Scripts/build_features.py
```

The resulting `03_Data/processed/features.csv` should contain $\geq$ 5,000 daily rows spanning 2011–present.

A.6 Step 5 — Train Models and Register with MLflow

Start MLflow tracking server first:

```
mlflow server --host 0.0.0.0 --port 5000 &
```

Train all three model families:

```
# LightGBM champion (t+1, t+3, t+7) - ~3 min on modern CPU
python 04_Scripts/train_lightgbm.py

# XGBoost strong-secondary (t+1, t+3, t+7) - ~2 min on modern CPU
python 04_Scripts/train_xgboost.py

# LSTM comparison (t+1, t+3, t+7) - ~60 min on CPU, ~15 min on GPU
python 04_Scripts/train_lstm.py
```

Inspect runs at http://localhost:5000. LightGBM models are automatically promoted to `Production` stage as the serving champion. XGBoost is loaded in parallel as a strong secondary (`?model=xgboost`). LSTM is logged as `Staging` for comparison only.

A.7 Step 6 — Run Analysis Scripts (Optional)

```
# Scenario C: alert threshold optimisation + PR curves
python 04_Scripts/scenario_c_threshold.py

# SHAP feature importance across all horizons
python 04_Scripts/shap_analysis.py
```

Figures are saved to `03_Data/results/` and `07_Final/report/figures/`.

A.8 Step 7 — Run Inference and Upload to S3

```
# Generate latest forecasts from the feature matrix
python 04_Scripts/run_inference.py

# Upload data, models, and results to S3
python 04_Scripts/upload_to_s3.py
# Or for a dry run to see what would be uploaded:
python 04_Scripts/upload_to_s3.py --dry-run
```

A.9 Step 8 — Build and Deploy Docker Container

```
# Build the slim serving image (~300 MB)
docker build -t caas-api:latest .

# Test locally
docker run -p 8000:8000 \
  -e AWS_REGION=ap-southeast-1 \
  -e S3_BUCKET_NAME=caas-mlops-bucket \
  caas-api:latest

# Verify API is running
curl http://localhost:8000/health
curl http://localhost:8000/forecast
```

For production deployment, SSH into the EC2 instance (provisioned in Step 3) and run the same Docker commands. The Terraform user-data script handles this automatically on first launch.

A.10 Step 9 — Enable CI/CD Automation

Add the following GitHub Actions secrets to the repository (`Settings -> Secrets and variables -> Actions`):

- `FIRMS_MAP_KEY` — NASA FIRMS API key
- `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`
- `S3_BUCKET_NAME` — `caas-mlops-bucket`
- `EC2_HOST` — public IP of the EC2 instance
- `EC2_SSH_KEY` — private SSH key content
- `MLFLOW_TRACKING_URI` — `http://<ec2-ip>:5000`
- `GITHUB_TOKEN` — automatically available in Actions

Once secrets are set, the three workflows activate automatically:

- **test.yml** — runs on every push to `main`
- **daily_pipeline.yml** — runs at 06:00 Bangkok time (23:00 UTC)
- **retrain.yml** — triggered by drift alert or December 1 schedule

A.11 Step 10 — Launch the Streamlit Dashboard

```
cd 04_Scripts/serve
streamlit run dashboard.py
# Opens at http://localhost:8501
```

The dashboard reads the latest forecast from `03_Data/results/latest_forecast.json` (updated daily by the pipeline). If the file is absent, it falls back to a demo mode with synthetic data.

A.12 Step 11 — Run the Test Suite

```
pytest                      # Runs all 45 tests with -v --tb=short
pytest tests/test_api.py    # API endpoint tests only
pytest tests/test_models.py  # Model file + inference tests
```

All 45 tests should pass. Tests that require model files will skip gracefully if models are not yet trained.

Appendix B

Full Metric Tables

B.1 LightGBM (Champion) — Validation and Test Metrics

Table B.1: LightGBM Champion Full Metrics (Validation and Test Sets)

Split	**Horizon**	**MAE**	**RMSE**	**R^2**	**AUROC**
Validation	t+1	7.71	—	—	—
Validation	t+3	9.12	—	—	—
Validation	t+7	9.72	—	—	—
Test	t+1	5.12	9.00	0.841	0.992
Test	t+3	6.77	11.78	0.729	0.973
Test	t+7	8.25	14.36	0.600	0.946

Validation MAE is the best-trial objective from the Optuna hyperparameter search (103 trials at t+1/t+3, 100 at t+7); the full regression suite (RMSE/R^2/AUROC) is recomputed only at test time.

B.2 XGBoost (Strong Secondary) — Validation and Test Metrics

Table B.2: XGBoost Secondary Full Metrics (Validation and Test Sets)

Split	**Horizon**	**MAE**	**RMSE**	**R^2**	**AUROC**
Validation	t+1	7.52	—	—	—
Validation	t+3	8.81	—	—	—
Validation	t+7	10.20	—	—	—
Test	t+1	5.31	8.95	0.843	0.992
Test	t+3	7.08	11.90	0.723	0.973
Test	t+7	8.90	15.32	0.544	0.933

B.3 LSTM — Validation and Test Metrics

Table B.3: LSTM Full Metrics (Validation and Test Sets)

Split	Horizon	MAE	RMSE	R^2	AUROC
Validation	t+1	7.12	11.45	0.730	0.978
Validation	t+3	9.45	16.20	0.512	0.914
Validation	t+7	9.87	16.78	0.543	0.932
Test	t+1	6.67	11.64	0.753	0.984
Test	t+3	8.87	15.89	0.540	0.927
Test	t+7	8.06	14.49	0.608	0.961

B.4 Alert Classification Metrics (LightGBM Champion, 50 µg/m³ threshold)

Table B.4: Binary Alert Classification Performance on Test Set (LightGBM Champion)

Horizon	Precision	Recall	F1	AUROC
t+1	0.800	0.911	0.852	0.992
t+3	0.713	0.785	0.747	0.973
t+7	0.704	0.722	0.713	0.946

For reference, the XGBoost strong-secondary alert performance is F1 = 0.844 / 0.723 / 0.584 at t+1/t+3/t+7 using the same 50 µg/m^3 threshold — LightGBM's higher recall discipline at t+7 is the main reason it holds the championship.

B.5 SHAP Mean Absolute Values — Top 5 Features per Horizon (XGBoost basis)

Table B.5: Top-5 SHAP Features by Horizon (Mean |SHAP| on Test Set)

Horizon	**Feature**	**Mean |SHAP|**
t+1	pm25_lag1	6.85
t+1	lag1_x_temp	3.64
t+1	precipitation	1.41
t+1	hotspot_7d_roll	1.31
t+1	sin_doy	0.89
t+3	hotspot_14d_roll	4.63
t+3	pm25_lag1	2.21
t+3	sin_month	2.14
t+3	day_of_year	2.08
t+3	hotspot_7d_roll	1.50
t+7	sin_month	4.63
t+7	hotspot_7d_roll	3.00
t+7	day_of_year	2.23
t+7	hotspot_14d_roll	2.23
t+7	week	2.15

SHAP values are computed on the XGBoost strong-secondary (gradient-boosted regime representative); see §3.8 for the rationale.

VITA

Supanut Kompayak

Student ID	st126055
Programme	Master of Science in Data Science and Artificial Intelligence
Specialisation	Data Engineering and Machine Learning Operations (MLOps)
Institution	Asian Institute of Technology (AIT), Pathum Thani, Thailand
Cohort	January 2026
Email	st126055@ait.asia
Research Interest	End-to-end MLOps pipelines, time-series forecasting, cloud-native machine learning systems, drift monitoring, and applied AI for environmental and public-health problems.

Contributions to CAAS: overall system architecture, data engineering pipeline (PM2.5, weather, FIRMS ingestion and feature engineering), MLOps stack design (MLflow, FastAPI, Evidently, Terraform), CI/CD workflows, AWS deployment, dashboard implementation, and final report preparation.

Shuvam Shrestha

Student ID	st125975
Programme	Master of Science in Data Science and Artificial Intelligence
Specialisation	Data Engineering and Machine Learning Operations (MLOps)
Institution	Asian Institute of Technology (AIT), Pathum Thani, Thailand
Cohort	January 2026
Email	st125975@ait.asia
Research Interest	Machine learning model development, gradient-boosted methods, deep sequence models, interpretability, and comparative model evaluation for time-series problems.

Contributions to CAAS: LightGBM champion development and Optuna hyperparameter tuning, XGBoost strong-secondary training and ablation study (Scenario D), LSTM architecture search and comparison, SHAP interpretability analysis, Scenario C alert-threshold optimisation, and evaluation-figure generation.